Multi-Protocol Peer-to-Peer Link Layer for Distributed GLP

Plan-stage Co-Design Dossier (feature 025) - illustrations, answers, and open questions

2026-06-06

Contents

0. Status and scope

This dossier collates the plan-stage co-design for feature 025 (the multi-protocol peer-to-peer link layer for distributed GLP): the rulings made so far, every code illustration, the end-to-end answers/explanations, and all open questions.

EVERYTHING marked PROPOSED is pending Gabi's language-authority approval (CLAUDE.md Language Authority; DISCIPLINE 1.14). The marathon plan gate is in progress; this is the working design surface, not a ratified spec. Full per-topic contracts live under `specs/025-multi-protocol-link-layer/contracts/` (`link-primitives.md`, `guards.md`, `architecture-context.md`, `codesign-proposal.md`, `example-http-link.md`, `rulings-log.md`).

GLP invariants are preserved exactly and are non-negotiable: SRSW (one reader / one writer per variable per clause, never relaxed by a flag), writer-MGU (binds only writers), three-valued unification (an un-arrived remote value behaves as an unbound local reader -> Suspend, never a spurious Fail), suspend-on-reader / reactivate-on-bind, bind-once monotonicity, per-link FIFO.

---

1. Rulings to date (Gabi)

Date	Item	Ruling
2026-06-06	Peer-id ordering (FR-037)	**B** - peer-ids MAY be non-numeric compound terms requiring a total order; the `@< @> @=< @>=` family is IN SCOPE. (Committed in spec.md.)
2026-06-06	Sender kernel (OQ-1/OQ-3)	**`'_link_send'/3` body-kernel APPROVED** ("sound"). Channel-face `link_send/3` is the idiomatic data path; `'_link_send'/3` backs the LinkId-keyed `out_relay/3`. The receiver needs NO kernel (host ingress only).
2026-06-06	Close (9th primitive)	**`link_close/1 (and /2)` APPROVED** - a NEW host system-predicate `'_link_close'` for ABRUPT teardown + to back the element-level sugar. Graceful close stays stream-end `[]`.
2026-06-06	Clean-close monitor term	**APPROVED** - a clean close emits a terminal `closed(LinkId, Reason)` term on the monitor stream (distinct from `tempFail`/`permFail`).
2026-06-06	CoAP reliability + DTLS	**ACCEPTED** - CoAP CON (confirmable) for transport ack/retransmit; our seq/dedup does ordering + dedup on top; DTLS = the secure variant.
2026-06-06	MQTT framing	**CLARIFIED** - at THIS level every link is peer-to-peer to the immediate peer; the MQTT broker is at ANOTHER level, OUT OF SCOPE here.
(prior, RULED in B2-B3-G doc)	Build target	C#-first reference, Dart mirror after; cross-runtime Dart<->C# parity required.
(prior)	Decomposition	One role-parameterized program (branch on ground AgentId), not a fork (FR-011).
(prior)	Failure model	Faults as bound terms on a per-link monitor stream (not a 4th verdict); lattice ok/tempFail/permFail; epoch/fencing for split-brain.
(prior)	T1 / T2	Broker = transport relay; keep BLE BIS multi-reader in scope as an open co-design item; broadcast = N bilateral ground-copy links.

2. The base link primitives (now NINE; all PROPOSED)

A link end is presented to GLP logic as a `Channel`, so the existing `self.glp send/receive/new_channel` idioms compose ABOVE the seam:

```
LinkId ::= link_id(Scheme, Endpoint, Nonce).        % ground, never reused
Link(In, Out) ::= ch(In, Out?).                      % a link end as a Channel
Fault ::= ok ; closed(LinkId, Reason)                % closed = intentional (clean)
        ; tempFail(LinkId, Reason) ; permFail(LinkId, Reason).
```

#	Primitive	Signature (modes)	One-line semantics	Language-authority
1	`link_setup/4`	`(LinkId?, LinkRole?, Link, FaultStream)`	establish-or-reuse a link by ground LinkId; idempotent (FR-007)	NEW pred `'_link_setup'/5`
2	`server_listener/3`	`(LinkId?, Link, FaultStream)`	establish by listening (FR-002 path A)	composable
3	`client_connector/3`	`(LinkId?, Link, FaultStream)`	establish by connecting (FR-002 path A)	composable
4	`request_link/4`	`(LinkId?, AgentId?, Link, FaultStream)`	establish by handshake - initiate (FR-002 path B)	NEW pred `'_link_request'/5`
5	`accept_link/4`	`(LinkId?, Stream(request)?, Link, FaultStream)`	establish by handshake - accept (FR-002 path B)	NEW pred `'_link_accept'/5`
6	`link_send/3` (+`out_relay/3`)	`(Term?, Link?, Link)` / `(Term?, LinkId?, AgentId?)`	ground-relay send (FR-010/040)	wrapper composable; LinkId-face backed by NEW kernel `'_link_send'/3`
7	`link_recv/3`	`(Term, Link?, Link)`	receive one element (suspend-not-fail)	composable (host ingress only)
8	`link_monitor/2`	`(LinkId?, FaultStream)`	per-link fault monitor (FR-008/043-047)	NEW pred `'_link_monitor'/2` + fault vocab
9	`link_close/1` (+`/2`)	`(LinkId?)` / `(LinkId?, Reason?)`	ABRUPT teardown (graceful = stream-end `[]`)	NEW pred `'_link_close'/2`

Key clauses (PROPOSED):

```
link_setup(LinkId, Role, ch(In?, Out), Faults?) :-
    ground(LinkId?), ground(Role?) | '_link_setup'(LinkId?, Role?, In, Out?, Faults).

accept_link(LinkId, [request(LinkId2, FromPeer)|_], ch(Link_In?, Link_Out), Faults?) :-
    ground(LinkId?), LinkId? =?= LinkId2? |
    '_link_accept'(LinkId?, FromPeer?, Link_In, Link_Out?, Faults).

link_send(Msg, ch(In, [Msg?|Out?]), ch(In?, Out)) :- ground(Msg?) | true.    % channel face
out_relay(Msg, LinkId, ToPeer) :-                                            % LinkId face
    ground(Msg?), ground(LinkId?), ground(ToPeer?) | '_link_send'(Msg?, LinkId?, ToPeer?).
```

```
link_recv(Msg?, ch([Msg|In], Out?), ch(In?, Out)).

link_monitor(LinkId, Faults?) :- ground(LinkId?) | '_link_monitor'(LinkId?, Faults).

procedure link_close(LinkId?).
procedure link_close(LinkId?, Reason?).
link_close(LinkId) :- ground(LinkId?) | '_link_close'(LinkId?, abrupt).
link_close(LinkId, Reason) :- ground(LinkId?), ground(Reason?) | '_link_close'(LinkId?, Reason?).
```

Verified correction (live code): the existing `'_send'/3` kernel (`body_kernels.dart:658-745`) ABORTS unless its 2nd arg is a `_w/2`/`_r/2` global name and then runs the madGLP globalize path - it is NOT a ground-relay, so it cannot back the base sender. Hence the NEW `'_link_send'/3` kernel (ground frame, no globalize).

---

3. The stream model: scalar -> stream -> bounded pipe

- **Scalar 1w/1r:** `producer(X)`, `consumer(X?)` - one value, bind-once.
- **Stream (unbounded pipe):** `Stream(T) ::= [] ; [T | Stream(T)]`. The producer conses `[V1,V2,... | Tail]`; each `Tail` is a FRESH 1w/1r cell. A stream is a chain of scalar cells. The consumer reads head-by-head and suspends on the unbound tail. A naive producer can run ahead unboundedly.
- **Bounded pipe (flow control):** couple a reverse CREDIT/demand stream; the producer must spend a credit per element and SUSPENDS (head unification on `[more|Credits]`) when none is left - pure suspend-on-reader, no buffer object:

```
Credit ::= more.

procedure produce(Stream(Item)?, Stream(Credit)?, Stream(Item)).
produce([Item|Items], [more|Credits], [Item?|Data?]) :- produce(Items?, Credits?, Data).
produce([], _, []).                                    % source done -> close data

procedure consume(Stream(Item)?, Stream(Credit)).
consume(Data, [more, more, more | Credits?]) :- drain(Data?, Credits).   % window of 3

procedure use(Item?).                                  % illustrative external sink (app handler)
procedure drain(Stream(Item)?, Stream(Credit)).
drain([Item|Data], [more | Credits?]) :- use(Item?), drain(Data?, Credits).
drain([], []).
```

Invariant: `items_produced - items_consumed <= N` (the window). In the link layer this is FR-025 backpressure, realized either BELOW the seam (the egress drainer's bounded queue + transport flow control - TCP window / HTTP/2 `WINDOW_UPDATE` / WS backpressure / CoAP CON acks - surfaced as producer suspension) or as a PROGRAM-VISIBLE credit back-channel (riding the link's reverse direction) for application-rate flow control. The credit stream IS the same reverse direction as the B->A back-channel.

> **KEY INSIGHT (HIGHLIGHTED) - flow control and the bidirectional link are ONE mechanism.** A bounded pipe over a link is just the forward data stream coupled to a reverse credit stream, and "bounded" is GLP suspension, not a buffer object. The credit/demand stream is the SAME reverse direction as the B->A back-channel - so application replies and flow-control credits ride one mechanism. This is potentially a HUGE benefit (a major simplification of flow control + back-channel into a single dataflow construct).
>
> [**NEEDS FURTHER ELABORATION**] (Gabi, 2026-06-06). Flagged for deeper design work (OQ-F3): the exact coupling of logical-term credits to byte-chunk credits; the fragmentation interaction; the per-scheme mapping; and whether one back-channel multiplexes both B->A data AND credits or uses separate streams.

Credit granularity (two coupled levels). (Confirms Gabi's model: per-byte-chunk, max for safety, min one byte.)

- **Logical (GLP) credit** = one stream term/element (`more` = permission for one element). Bounds the number of in-flight TERMS. This is what a GLP program sees.
- **Transport (byte) credit** = per byte-stream chunk, BELOW the seam (HTTP/2 `WINDOW_UPDATE` is byte-based; CoAP blockwise is block-based; TCP window is byte-based; WS via socket backpressure). Bounds in-flight BYTES.
- They COMPOSE: a large term fragments (FR-022) into chunks, each byte-credited, so one logical credit can consume several byte-chunk credits.
- **Maximum chunk applies (safety):** bounded so no oversized allocation - over-MTU frames fragment; malformed/oversized/huge-arity frames fail safe within bounded memory and stack (FR-022/FR-028).
- **Minimum one byte:** a credit always grants at least one byte of forward progress, so a non-empty chunk can always advance - no zero-window deadlock (cf. TCP zero-window probe / silly-window-syndrome avoidance).

---

4. Link lifecycle: establish once, send/receive repeat, then close

1. **Establish (once):** `server_listener/client_connector` (path A) or `request_link/ accept_link` (path B). 2-3. **Send/receive (1..N times):** a link end is a stream; `link_send/link_recv` (or direct stream recursion) repeat until the stream ends.
2. **Close:**
   - **Graceful (default): stream-end `[]`.** The producer binds its `Out` tail to `[]`; the consumer's `consume([])` fires. No primitive needed.
   - **Abrupt: `link_close(LinkId)`** (the 9th primitive) - tear down regardless of stream state (early-stop / fault give-up / security kill), and the face for the element-level `link_send/link_recv` sugar (which hides the `[]`).
   - Either way the host runs per-link GC (FR-024) and emits a terminal `closed(LinkId, Reason)` (Reason = `eos` for graceful, the user reason for `link_close`) on the monitor, then ends the monitor stream. A disconnect that is NOT an intentional close yields `tempFail` then `permFail` - never a logical Fail.

---

5. Worked example: producer/consumer split over HTTP

One role-parameterized program (FR-011); both nodes load it and boot with their ground AgentId. The shared variable X becomes a link.

```
-module(http_split_demo).

% "https" because the ends are on different hosts (FR-029 refuses plain http inter-host).
procedure demo_link(LinkId).
demo_link(link_id("https", ep("nodeB.example", 8443), 1)).

procedure main(AgentId?).
main(Me) :- Me? =?= producer |
    demo_link(L), client_connector(L?, Link, Faults), run_producer(Link?, Faults?).
main(Me) :- Me? =?= consumer |
    demo_link(L), server_listener(L?, Link, Faults), run_consumer(Link?, Faults?).

procedure run_producer(Link(_,_)?, FaultStream?).
run_producer(ch(_, Out?), _) :- produce([10, 20, 30], Out).
procedure produce(Stream(Integer)?, Stream(Integer)).
```

```
produce([V|Vs], [V?|Out?]) :- ground(V?) | produce(Vs?, Out).
produce([], []).                                        % graceful close (outbound [])

procedure run_consumer(Link(_,_)?, FaultStream?).
run_consumer(ch(In, []), _) :- consume(In?).
procedure consume(Stream(Integer)?).
consume([V|In]) :- ground(V?) | use_value(V?), consume(In?).
consume([]).                                            % close detected (inbound [])

procedure use_value(Integer?).
use_value(V) :- ground(V?) | '_output'(V?).
```

End-to-end:

1. **Boot** - same source; B booted `consumer`, A booted `producer`; `Me? =?= producer` selects the role clause (ground-AgentId, three-valued).
2. **Establish** - B `server_listener` -> `'_link_setup'` starts the HTTPS server on nodeB:8443, registers the link by ground LinkId, mints In/Out + Faults, installs the host ingress (fills In) and egress drainer (drains Out); `consume` SUSPENDS on empty In. A `client_connector` -> `'_link_setup'` dials nodeB:8443 (TLS). Same ground LinkId => one bilateral link (FR-002/005). Establishment role is independent of data direction (FR-004).
3. **Send** - A `produce` conses `42` etc. onto `Out` in the HEAD; `ground(Msg?)` guarantees no placeholder/embedded reader crosses (FR-010). The egress drainer serializes (byte-parity serializer + seq/version/CRC frame) and POSTs to B.
4. **Receive** - B's ingress deserializes, runs the dedup gate (FR-021), binds the In tail (writer-MGU on B's LOCAL writer, FR-049), which reactivates the suspended `consume` exactly once (FR-051). B prints 42 - byte-identical to the unsplit run (SC-001).
5. **Close** - A's `produce([], [])` ends Out -> END_STREAM -> B's `consume([])` fires; both directions ended -> host GC -> `closed(LinkId, eos)` on the monitor.

Switching `"https"` to `"wss"`/`"mqtt"`/`"coap"` changes ONLY the `Scheme` in `LinkId`; the GLP program is unchanged (FR-006/013).

---

6. Transport leaves (corrected per the MQTT clarification)

At this level every link is peer-to-peer to the immediate peer. Brokers/relays are at another level, out of scope here. What differs per protocol is how the leaf maps the uniform seam (open / send-bytes / recv-bytes / close + fault) and how it carries the reverse (B->A) back-channel.

- **WS / WSS** - persistent full-duplex socket; In/Out map straight onto the two directions; the listener (server) pushes B->A frames natively. `wss` = WS over TLS. The natural fit.
- **HTTPS / mTLS** - HTTP over TLS. Over HTTP/2: one long-lived bidirectional stream (client DATA = A->B, server DATA = B->A). **mTLS = mutual TLS:** the listener requires+verifies the client cert and the connector presents one -> both ends cryptographically authenticated, serving FR-029 (TLS-by-default) and strengthening FR-026 (origin authentication). Over HTTP/1.1 the back-channel uses response bodies / long-poll.
- **MQTT** - the immediate link is still peer-to-peer (e.g. peer<->broker). The broker, if present, is a SEPARATE node at ANOTHER level (its own P2P links to subscribers) and is OUT OF SCOPE for the base link primitives. The base primitive just sees one bilateral P2P link to its immediate peer; any fan-out/forwarding through a broker is a higher-level concern (routing/glink-like), not modelled here.
- **CoAP** - UDP, REST-like, constrained. A->B via PUT/POST; B->A back-channel via OBSERVE (server push, RFC 7641). Small MTU (~1 KB) => blockwise transfer (RFC 7959) drives the reliability sublayer's fragment/reassemble (FR-022). CON (confirmable) messages for transport ack/retransmit; our seq/dedup on top (FR-020/021). DTLS = the secure variant.

---

7. The request/accept handshake establishment path (FR-002 path B)

Used when there is no direct listen/connect (NAT traversal, discovery, peer introduction).

1. **A initiates** `request_link(LinkId, B, Link, Faults)` -> `'_link_request'` sends a ground `request(LinkId)` token to B over a rendezvous, mints A's In/Out, and parks (A's recv suspends).
2. **B accepts** - B reads its inbound RequestStream; `accept_link(LinkId, RequestStream, Link, Faults)` matches `request(LinkId2, FromPeer)` by `LinkId? =?= LinkId2?` (existing guard, three-valued) and establishes via `'_link_accept'`.
3. **Convergence** - both route through the SAME `'_link_setup'` registry keyed by ground LinkId, so the resulting Link is indistinguishable from the listen/connect path ("equivalent established link", FR-002). Then send/receive/close are identical.

Rendezvous (OQ): in-band over the transport connect (recommended), a pre-established bootstrap link, or a discovery service.

---

8. Guard set (PROPOSED; against docs/guards-reference.md)

- **ADD** `@< @> @=< @>=` (standard-order over GROUND terms; suspend-until-ground; ground-implying for SRSW). Multi-site core edit: lexer (`@` lookahead vs `Goal@Agent`) + token + parser + runner `_evaluateGuard` + SRSW analyzer + prelude.
- **FIX** `atom/1` (runner has no arm -> warn+fail today, but analyzer accepts+grounds it).
- **FIX** compound-operand-suspend (`_dereferenceWithTracking` must recurse into compound args, mirroring the correct GroundEqual recursion) - today a nested unbound reader wrongly FAILs (FR-034).
- **FIX** imported-reader-reactivation (`handleMadAssignment` must route imported-reader cells through `bindImportedReader`) - today such suspensions never wake (FR-035). [OPEN: D-B2-3 alternative - rule the link layer to local-pair writers only.]
- **DECLINE** `== \== \= reader/1` (redundant aliases / non-monotone). **Leave `=\=` untouched** (load-bearing).

Every new/changed guard must show three-valued ask-semantics (succeed / suspend-then- reactivate-once / fail) as REPL Section-A runtime + Section-B/C type-check tests, with the baseline suite green before/after (FR-039/067).

---

9. Architecture context (summary)

- **C#-first reference** (out/csharp, the mandated-default REPL; payload_serializer.cs is byte-parity with Dart); Dart mirror authored AFTER the C# reference works (FR-055/056). Hand-authored C# lives in a clobber-safe home OUTSIDE out/csharp and glp_runtime_net so a codeconv regen cannot overwrite it (FR-057).
- **Uniform LinkTransport seam** (open / send-bytes / recv-bytes / close + fault) selected by Scheme; per-protocol leaves behind it, per-platform, not auto-converted (FR-058).
- **Reliability sublayer** (the real net-new engineering): per-link sequence/dedup, FIFO + reorder buffer, idempotent redelivery (today a duplicate frame CRASHES the agent - verified - must become an absorbed no-op, FR-021), serializer cycle-guard + version byte + length/CRC + fragmentation, epoch/fencing (split-brain), distributed GC, reply-table/CorrId, bounded backpressure. Byte/behaviour-identical Dart<->C# (FR-060/061).
- **Failure model** (above): faults are bound terms on the monitor stream; ok / closed / tempFail / permFail; disconnect never -> Fail.
- **Cross-runtime parity gate** (FR-059/062): one program split Dart-instance <-> C#-instance over one link, equivalent to unsplit; an executed Dart<->C# round-trip MUST pass before ship.

10. ALL open questions (consolidated)

Primitives / establishment - OQ-A1: single bidirectional `Link(In,Out)` vs separate sender/receiver handle types (recommend single bidirectional). - OQ-A2: LinkId identity - ground compound `link_id(Scheme, Endpoint, Nonce)` (so `=?=`/`@<` test it with no new machinery - recommended) vs opaque host handle. - OQ-A3: request/accept rendezvous - in-band over connect (recommended) vs bootstrap link vs discovery service. - OQ-A4: ratify exact names/arities for the NEW predicates `'_link_setup'/5`, `'_link_request'/5`, `'_link_accept'/5`, `'_link_send'/3`, `'_link_monitor'/2`, `'_link_close'/2`.

Close / monitor - OQ-C1: ratify `link_close/1` + `/2` shape and the `'_link_close'(LinkId?, Reason?)` kernel. - OQ-C2: ratify the clean-close term name `closed(LinkId, Reason)` (vs `bye`) and the `eos` reason for graceful close.

Flow control - OQ-F1: default per-link window N (the drainer's bounded-queue depth) - fixed / scheme-defaulted / program-settable? - OQ-F2: expose a program-visible credit back-channel in the MVP, or below-seam backpressure only to start? - OQ-F3 [NEEDS ELABORATION - flagged by Gabi as a potentially HUGE benefit]: the credit/demand stream = the B->A back-channel unification. Elaborate: logical-term vs byte-chunk credit coupling; fragmentation interaction with credit accounting; whether the GLP program sees only logical credits (recommended) with byte credits below the seam; per-scheme mapping (HTTP/2 WINDOW_UPDATE / CoAP blockwise / WS backpressure / TCP window); and whether ONE back-channel multiplexes both B->A data and credits or uses separate streams. (Max chunk for safety; min one byte for progress.)

Guards - OQ-G1: `@<` total order (proposed Number < String < compound, then arity/functor/args) - confirm, and that it is byte/behaviour-identical Dart<->C# (FR-060). - OQ-G2: `@<` family negatable or non-negatable (proposed non-negatable, complement `@>=`). - OQ-G3: `atom/1` exact semantics - exact synonym of runtime `string/1` (excludes `[]`/`nil`), or also accept `[]`? - OQ-G4 (D-B2-3): imported-reader fix - wire `handleMadAssignment` -> `bindImportedReader` (fixes the latent core hazard for later glink) vs rule the link layer to local-pair writers only (sufficient for the ground-relay MVP). Changes the fix entirely. - OQ-G5 (SC-017): the `=\=`-gated division/mod guarantee targets which prelude? (no `=\=` occurrence exists in programs/self.glp today.)

Transport / scope - OQ-T1: BLE LE-Audio BIS true-multi-reader vs SRSW - kept in scope as a later co-design item (confirm it stays, not dropped). - OQ-T2: per-platform matrix (T4) - which leaf on Windows vs Android.

Non-language (for the eng/implementation gate, not the language decision) - The reliability sublayer is substantial net-new work below the seam (0 hits today). - The FR-021 duplicate-delivery crash is LIVE and must be fixed in lockstep with `link_recv`. - The C# host predicates need the clobber-safe home (FR-057); the seam's async recv signature is a codeconv-escalation item. - No deterministic distributed/real-transport test harness exists yet - integration testing is net-new (see the tutorial+test plan).

11. Invariants preserved (checklist)

Invariant	How
SRSW per instance (FR-048)	each side has its own local pair; every clause hand-checked
writer-MGU (FR-049)	ingress binds only the local In-tail writer
three-valued / suspend-not-fail (FR-017/050)	recv suspends on unbound In head
reactivate exactly once (FR-051)	one ingress bind wakes the suspended recv once
bind-once monotonic (FR-052)	dedup gate makes a redelivered frame a no-op
per-link FIFO (FR-018/053)	Out cons order = wire send order = In bind order
ground-relay (FR-010/040)	`ground(Msg?)` gate in `link_send`

Invariant	How
faults are data (FR-043)	ok / closed / tempFail / permFail terms on the monitor stream
one program, not a fork (FR-011)	single `main/1`, role by ground AgentId